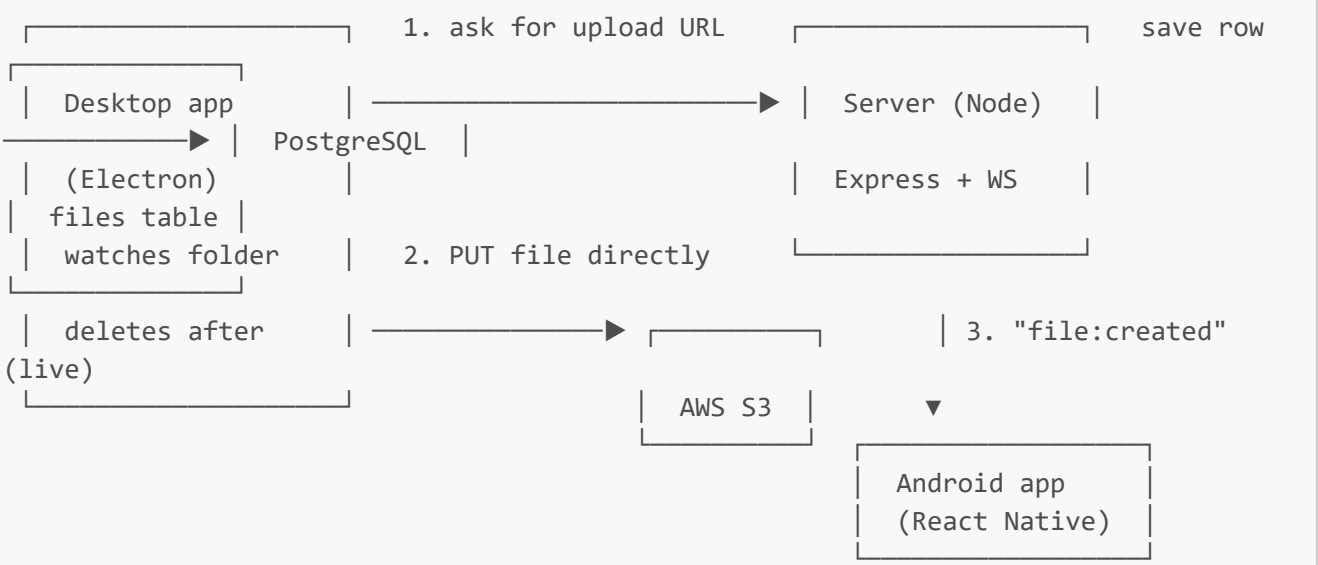
CloudSync: Laptop folder → AWS S3 → Android app

Put a file in a folder on your laptop. It uploads to S3, its URL is saved in your database, the local copy is deleted, and the file shows up on your Android phone within a few seconds.



Folder	What it is
`database/`	`schema.sql`, the 3 tables
`server/`	API + S3 + real-time updates. **Only this part holds your AWS keys.**
`desktop/`	Windows app: pick folders, watch them, upload, delete local copy
`mobile/`	React Native (Expo) Android app: browse, search, open, share, delete, upload from phone

Features

- **Instant sync:** new files are picked up as soon as they finish copying or downloading (the app waits until the file stops growing).
- **Existing files too:** anything already in the folder when the app starts gets uploaded.
- **Subfolders kept:** `Invoices/2026/Oct/a.pdf` keeps its path in S3 and in the app.
- **Duplicate detection:** a SHA-256 hash stops identical files from being uploaded twice.
- **Safe delete:** the local file is deleted only after S3 confirms the upload with the right size, and never if the file changed during upload.
- **Automatic retries** with back-off (5s → 5min) when the internet drops. Unfinished files stay in the folder and resume on the next start.
- **Ignores temporary files** such as `.crdownload`, `.part`, `.tmp`, Office `~$` lock files, and `desktop.ini`.
- **Live phone updates:** new files appear with a "NEW" badge. You can see whether the laptop is online and watch upload progress live.
- **Mobile:** search, filter by type (images/videos/docs…), image thumbnails, open, download, share link, delete, and upload from the phone.

- **Activity log** (`sync_events` table) shown in the app.
- **Private bucket:** files are viewed through temporary signed links (1 h). AWS keys never leave the server.
- Multiple folders, pause/resume, parallel uploads, and an optional start with Windows.

Database tables

The full SQL is in database/schema.sql. Summary:

`sync_folders`: folders being watched

column	type	note
id	UUID PK	
name	VARCHAR	"Invoices"
device_name	VARCHAR	laptop name
local_path	TEXT	`D:\Scans\Invoices` (unique per device)
is_active	BOOLEAN	
file_count	INT	
last_synced_at, created_at	TIMESTAMPTZ	

`files`: one row per file in S3 (the main table)

column	type	note
id	UUID PK	
folder_id	UUID FK → sync_folders	
file_name	VARCHAR	`report.pdf`
relative_path	TEXT	`2026/Oct/report.pdf`
extension, mime_type	VARCHAR	
category	VARCHAR	image / video / audio / document / archive / other
size_bytes	BIGINT	
sha256	CHAR(64)	duplicate detection
s3_bucket	VARCHAR	
s3_key	TEXT UNIQUE	path inside the bucket
s3_url	TEXT	`https://bucket.s3.region.amazonaws.com/...`
status	VARCHAR	uploading / uploaded / failed / deleted
source, source_device	VARCHAR	desktop or mobile, and which device

column	type	note
local_modified_at, uploaded_at, deleted_at, created_at	TIMESTAMPTZ	

`sync_events`: activity log (uploaded, duplicate_skipped, upload_failed, deleted, folder_added, folder_removed)

Step-by-step setup (beginner friendly)

You need **Node.js 18+** (you have v22) and an **Android phone**.

Step 1: Create the S3 bucket (AWS Console)

1. Sign in at https://console.aws.amazon.com and choose a region at the top right, e.g. **Asia Pacific (Mumbai)** `ap-south-1`.
2. Open **S3**, click **Create bucket**, and give it a globally unique name, e.g. `sami-cloudsync-files`.
3. Leave **Block all public access = ON** (files stay private; the app uses signed links). Click **Create bucket**.

Step 2: Create an AWS access key that can only use this bucket

1. Open **IAM** → **Users** → **Create user**, with name `cloudsync-server`. Do not give it console access.
2. Under **Permissions** → **Attach policies directly** → **Create policy** → **JSON**, paste the policy below (replace the bucket name) and save it as `CloudSyncS3Access`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::sami-cloudsync-files/*"
    }
  ]
}
```

3. Attach that policy to the user and finish creating the user.
4. Open the user, go to **Security credentials** → **Create access key** → **"Application running outside AWS"**. Copy the **Access key ID** and **Secret access key**. The secret is shown only once.

Step 3: Create the PostgreSQL database

Pick **one** option:

- **Option A, free cloud database (recommended; it also works when you deploy later):** sign up at https://neon.tech, create a project, and copy the **connection string** (`postgresql://user:pass@ep-`

`xxx.neon.tech/neondb?sslmode=require`). You'll set `DATABASE_SSL=true`.

- **Option B, on your laptop:** install PostgreSQL from https://www.postgresql.org/download/windows/ (remember the password you set), then open **SQL Shell (psql)** and run `CREATE DATABASE cloudsync;`. Your URL is `postgresql://postgres:YOURPASSWORD@localhost:5432/cloudsync`.

Step 4: Start the server

```
cd C:\syncing\cloudsync\server
copy .env.example .env
notepad .env
```

Fill in `.env`:

- `API_KEY`: make up a long random password, e.g. `cs_7f3k9Qm2xLp8Rz4vN1`. The desktop and phone apps use this same value.
- `DATABASE_URL` and `DATABASE_SSL` from Step 3.
- `AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_BUCKET` from Steps 1–2.

Then run:

```
npm install          # already done once, harmless to repeat
npm run db:init      # creates the 3 tables -> "Database tables are ready"
npm start            # -> "CloudSync server running on http://localhost:4000"
```

Check it by opening http://localhost:4000/api/health in a browser. You should see `{"ok":true,"database":"up",...}`. Keep this window open while you use the apps.

Step 5: Start the desktop app (laptop)

Open a **second** terminal:

```
cd C:\syncing\cloudsync\desktop
npm install
npm start
```

In the CloudSync window:

1. **Settings:** set Server URL `http://localhost:4000` and paste your API key. Click **Save**, then **Test connection**. The badge at the top right turns green ("Connected").
2. **Synced folders:** click **Browse...** (or paste a path like `D:\ToCloud`) and then **Add**.
3. Copy any file into that folder. Within about 2 seconds it shows under **Transfers**, uploads, and disappears from the folder. Check the S3 console (the file is under `cloudsync/...`) and the database (`SELECT file_name, s3_url FROM files;`).

> To keep local copies, untick **Delete local file after upload**.

Step 6: Run the Android app

1. On the phone, install **Expo Go** from the Play Store.
2. Make sure the phone and laptop are on the **same Wi-Fi**.
3. Find the laptop's IP address: run `ipconfig` and look for **IPv4 Address** under Wi-Fi, e.g. `192.168.1.10`.
4. Allow the server port through Windows Firewall (run once in **PowerShell as Administrator**):

   ```
   New-NetFirewallRule -DisplayName "CloudSync 4000" -Direction Inbound -
   Protocol TCP -LocalPort 4000 -Action Allow
   ```

   Test it by opening `http://192.168.1.10:4000/api/health` in the phone's Chrome browser. You should see `"ok":true`.
5. On the laptop, open a **third** terminal:

   ```
   cd C:\syncing\cloudsync\mobile
   npm install
   npx expo start
   ```

6. Scan the QR code with **Expo Go**. In the app's **Settings**, enter `http://192.168.1.10:4000` and the API key, then tap **Save & connect**.
7. Drop a file into the laptop folder and watch it appear on the phone.

Step 7 (optional): Build a real APK to install

```
cd C:\syncing\cloudsync\mobile
npm install -g eas-cli
eas login                       # free Expo account
eas build:configure
eas build -p android --profile preview
```

EAS gives you a download link for the `.apk`. Before building, add this to `eas.json` under `build.preview`: `"android": { "buildType": "apk" }`. You already have the Android SDK, so `npx expo run:android` also works to build and install over USB.

Step 8 (optional): Use it from anywhere, not just home Wi-Fi

So far the server runs on your laptop, so the phone only works on the same Wi-Fi. To see files from anywhere:

- Deploy the `server/` folder to **Render.com** (Web Service, build `npm install`, start `npm start`, add the same `.env` values under Environment) or **Railway**, an **AWS EC2** instance, etc.
- Use a cloud database (Neon, from Step 3 Option A).
- Change the Server URL in **both** apps to the `https://...` address.

Daily use

1. Start the server (or skip this if it's deployed).
2. Open CloudSync on the laptop (tick **Start CloudSync when Windows starts** so it opens by itself).
3. Put files in the folder. They upload, get deleted locally, and appear on the phone.

Troubleshooting

Problem	Fix
Server exits with "Missing environment variables"	`.env` is missing or incomplete. See Step 4.
`/api/health` says `database: down`	Wrong `DATABASE_URL`/password, or Neon without `DATABASE_SSL=true`
Desktop shows "Wrong API key"	The key in the app must match `API_KEY` in `server/.env` exactly
Upload error `S3 rejected upload: AccessDenied`	Check the IAM policy's bucket name and the `S3_BUCKET` / `AWS_REGION` values
Phone: "Cannot reach server"	Same Wi-Fi? Firewall rule (Step 6.4)? Using the laptop IP, not `localhost`?
File not deleted locally	It's open in another program (Windows locks it). Close it; the log shows a warning.
Files over 5 GB	Not supported (S3 single-upload limit)

API reference (all routes need the header `x-api-key`)

Method	Path	Purpose
GET	`/api/health`	health check (no key needed)
GET	`/api/files?search=&category=&folderId=&limit=&offset=`	list files
GET	`/api/files/:id` · `/api/files/:id/url?download=1`	file details · fresh signed link
DELETE	`/api/files/:id`	delete from S3 (the row stays as history)
POST	`/api/uploads/init` → `/api/uploads/:id/complete`	upload flow
GET/POST/DELETE	`/api/folders`	watched folders
GET	`/api/stats` · `/api/events`	totals, online laptops · activity log

Real-time (Socket.IO) events: `file:created`, `file:deleted`, `event:new`, `agents`, `folders:changed`.